



Problem A. Alejandro, lee por favor

Source file name:	Alejandro.c, Alejandro.cpp, Alejandro.java, Alejandro.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/2/2 (C++/Java/Python) second(s) / 64 megabytes
Author(s):	Carlos Alberto Salazar Meza - UFPS Cúcuta Colombia

Milton, profesor de programación en Ingeniería de Sistemas en otras épocas, sospecha que buena parte de sus estudiantes está entregando informes escritos por una IA sin siquiera leerlos. Para probarlo, escondió una trampa: al final del enunciado del trabajo dejó un párrafo en letra blanca, invisible para cualquiera que abra el documento y lea normalmente, pero perfectamente legible para una IA a la que le copien y peguen todo el texto. Ese párrafo no hablaba del informe en sí: era una instrucción de programación disfrazada de “estilo de escritura”, que le pedía a quien generara el texto que fuera cifrando cada letra que escribiera, siguiendo un algoritmo muy particular.

Justo antes de empezar a recibir los informes, Milton les contó a todos sobre la trampa que había escondido y el algoritmo que usaba: se mantiene un contador global c , inicialmente 0, y un contador de apariciones para cada una de las 26 letras del alfabeto, todos inicialmente en 0. El texto se genera letra por letra, y cada letra L que se iba a escribir originalmente se reemplaza por la letra que está c posiciones adelante en el alfabeto, de forma cíclica (después de la ‘z’ se vuelve a la ‘a’). Inmediatamente después de escribir esa letra, se incrementa en uno el contador de apariciones de L (la letra original, no la que quedó escrita); y si ese contador de apariciones de L es ahora múltiplo de un número secreto K , el contador global c aumenta en uno.

Para comprobar quién había caído en la trampa, Milton empezó a llamar a estudiantes al azar para que leyeran su propio informe en voz alta frente a todo el curso: quien lo hubiera escrito de verdad no tendría ningún problema, pero a quien la IA le hubiera redactado (y de paso cifrado) el texto, le tocaría leer una sopa de letras sin sentido – con el algoritmo ya conocido por todos, cualquiera podría descifrarla con papel y lápiz, pero no de memoria y en tiempo real, frente a todo el curso. Cuando le llegó el turno a Alejandro, un estudiante del semillero de investigación SILUX, Milton lo miró fijo y le dijo: “Alejandro, lee por favor.” Lo que Milton no sabía es que Alejandro sí había usado la IA – pero por pura pereza: era un estudiante muy inteligente, perfectamente capaz de haber escrito el informe él mismo, simplemente no había querido –, así que mientras leía fue aplicando el algoritmo letra por letra en su cabeza, y recitó su informe perfectamente coherente, como si nada.

Gracias a la lectura perfecta de Alejandro, Milton quedó feliz y tranquilo, y decidió preguntarle a una sola persona más, solo por compromiso. Esa persona eres tú, y no quieres decepcionarlo. Tu informe – el que en realidad escribió la IA por ti – llega representado, ya cifrado, como una lista de N palabras (el texto fue limpiado de tildes, signos de puntuación y mayúsculas, y separado en palabras). ¿Eres lo bastante inteligente como para descifrarlo tan rápido como Alejandro?

Input

La primera línea contiene dos enteros N y K ($1 \leq N \leq 2 \cdot 10^5$, $1 \leq K \leq 10^6$), la cantidad de palabras del informe y el número secreto usado por el algoritmo de Milton.

Le siguen N palabras, formadas únicamente por letras minúsculas de la ‘a’ a la ‘z’ (cada palabra tiene entre 1 y 10^6 letras, y la suma de las longitudes de las N palabras no supera 10^6), las palabras del informe ya cifradas, en el mismo orden en que fueron escritas originalmente.

Output

Imprime las N palabras originales del informe, separadas por un espacio, en una única línea.

Example

Input
2 3 aaa bbb
Output
aaa aaa
Input
5 1 a b c d e
Output
a a a a a
Input
3 100 hola silux mundo
Output
hola silux mundo
Input
9 4 el algoritmo secreto de njmupp hwg fguewdkgtvr sru wmqad
Output
el algoritmo secreto de milton fue descubierto por silux

Use fast I/O methods

Explanation

En el primer ejemplo, $K = 3$. Al descifrar la primera palabra (“aaa”), el contador c empieza en 0, así que las tres letras quedan tal cual: “aaa”. Justo después de esa letra ‘a’ número 3 (múltiplo de $K = 3$), el contador sube a $c = 1$. Por eso, al descifrar la segunda palabra (“bbb”), cada letra se retrocede una posición en el alfabeto y se recupera “aaa” de nuevo.



Problem B. Betito el viajero

Source file name:	Betito.c, Betito.cpp, Betito.java, Betito.py
Input:	Standard
Output:	Standard
Time / Memory limit:	2/3/4 (C++/Java/Python) second(s) / 128 megabytes
Author(s):	Jorge Humberto Teran Pomier - UMSA Bolivia

“Betito” va de viaje en sus próximas vacaciones y quiere saber cuantos lugares puedes visitar. El país que visitará se llama Matrizlandia y ... ¡obviamente es de forma matricial compuesta por celdas! con R filas y C columnas.

“Betito” tiene un mapa de Matrizlandia en el cual tiene marcado con un punto (“.”) los lugares (las celdas) que puede visitar y con un numeral (“#”) los lugares que no puede visitar. De un lugar marcado con un punto se puede ir a otro lugar marcado con un punto siempre y cuando estos sean adyacentes, es decir, el segundo lugar se encuentre inmediatamente, ya sea a la derecha, a la izquierda, arriba o abajo del primer lugar.

“Betito” se encuentra en el lugar del mapa (es decir, en la celda de la matriz) marcada con el símbolo asterisco (“*”).

¿Cuántos son los lugares que “Betito” puede visitar en Matrizlandia incluyendo el lugar donde inicialmente se encuentra?

Input

La entrada consiste de múltiples casos de prueba (los cuales son como máximo 10). Cada caso de prueba comienza con una línea que contiene dos números enteros positivos R C ($2 \leq R, C \leq 1000$), los cuales representan respectivamente, las filas y las columnas de Matrizlandia. El caso de prueba continua con R líneas, cada una conteniendo C caracteres en el conjunto de símbolos $\{., \#, *\}$.

La entrada de datos finaliza con una línea que contiene 0 0, para la cual no se debe generar ninguna salida.

Nota: Se garantiza que por cada caso de prueba solamente hay un símbolo asterisco (“*”) con el cual se indica el punto de partida en cual inicia “Betito” su paseo.

Output

Para cada caso se prueba se debe imprimir en una línea un número que indica el total de lugares que “Betito” pueden visitar incluyendo el punto de partida en el cual comienza su paseo.



Example

Input	Output
7 7	22
#.#...#	44
#.#.#.#	43
#.#.##.	32
..*.#..	
#.#.##.	
#.#.#.#	
#.#...#	
7 11	
*.....	
#####.	
.....	
.#####	
.#...#....	
.#####.###.	
.....	
9 11	
.#.....	
..#####	
....#...#.	
...#.#.#.#.	
#.##.#.#.#.	
.#.#.#.#.#.	
.#...#.#...	
.###.#.###.	
.#.*...#.#.	
9 6	
.##.#.	
#..##.	
#.#...	
...#.#	
.#.#..	
..##.#	
#..#...	
.*.##.	
.#....	
0 0	

Use fast I/O methods



Problem C. Company

Source file name:	Company.c, Company.cpp, Company.java, Company.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/3/4 (C++/Java/Python) second(s) / 64 megabytes
Author(s):	Santiago Guaneme - IOI Colombia

Actualmente existe una compañía en la que cada miembro tiene un jefe directo y puede tener cero o más empleados directos, excepto el dueño de la empresa (el nodo 1) que no tiene jefe.

Por preferencias del dueño, dentro de esta compañía, 2 miembros pueden comunicarse si y sólo si uno de ellos es el jefe directo del otro (no se considera jefe directo al jefe de mi jefe), es decir que si el miembro U necesita al miembro V , debe llamar al miembro adyacente en la jerarquía que lo acerque al miembro requerido, pasarle la información, y ahora este miembro debe seguir el mismo procedimiento. En base a las condiciones dadas se puede determinar que la jerarquía forma un árbol cuya raíz es 1, por lo que hay un camino único entre cada par de nodos.

Dada esta forma tan impráctica de comunicarse, la cantidad de llamadas realizadas está siendo bastante costosa para la empresa, por lo que piden tu ayuda para determinar dos valores. Primero, la suma de las distancias entre todos los pares de nodos, es decir $\sum \text{dist}(u, v)$, para todo $1 \leq u < v \leq N$, y segundo, definamos $F(X)$ como el diámetro del subárbol con raíz en X . Se pide determinar $F(X)$ para todo X .

$\text{dist}(u, v)$ representa la cantidad de llamadas realizadas entre u y v , en otras palabras, la cantidad de aristas en su camino único.

El subárbol de un nodo X está formado por X y todos sus descendientes.

El diámetro de un árbol es la máxima distancia entre cualquier par de vértices.

Input

En la primera línea un entero $2 \leq N \leq 10^6$, En la segunda línea se reciben $N - 1$ enteros $P_2, P_3, \dots, P_N$ donde P_i es el jefe directo (padre) del nodo i , se garantiza que $P_i < i$.

Output

Primero se debe mostrar la suma de $\text{dist}(u, v)$ para cada $u < v$, en la segunda línea mostrar N elementos $F(1), F(2), \dots, F(N)$.

Example

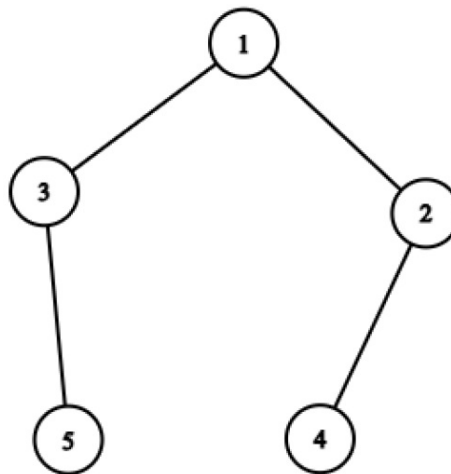
Input	Output
5 1 1 2 3	20 4 1 1 0 0

Use faster I/O methods¹

¹Por ejemplo, para C++ colocar al inicio del **main** las líneas:
`ios_base::sync_with_stdio(false);`
`cin.tie(0);`
`cout.tie(0);`

Explanation

Para el ejemplo del caso de prueba se tiene:



El diámetro del subárbol del nodo 1 es 4, en el 2 y en el 3 es 1, y en el 4 y 5 es 0, para la suma de distancias, tenemos $4(4, 5) + 3(4, 3) + 3(5, 2) + 2(4, 1) + 2(5, 1) + 2(2, 3) + 1(2, 1) + 1(3, 1) + 1(5, 3) + 1(4, 2) = 20$.



Problem D. Dangerous odyssey

Source file name:	Dangerous.c, Dangerous.cpp, Dangerous.java, Dangerous.py
Input:	Standard
Output:	Standard
Time / Memory limit:	2/3/6 (C++/Java/Python) second(s) / 128 megabytes
Author(s):	Roberto Solís - UAZ México

El planeta E-13 orbita alrededor de una estrella en una galaxia lejana llamada UAZ. En ese planeta, ha tenido lugar en los últimos 10 años una gran batalla en la isla de Yorta, en la cual el rey Osideo ha logrado la victoria. Ahora desea emprender su camino de regreso a su amado reino, la isla de Acati, utilizando su embarcación. Sabe que en el trayecto hay lugares donde habitan sirenas cuyos cantos alcanzan una distancia D_c desde los lugares donde habitan y brujas cuyos hechizos que también alcanzan una distancia D_h desde los lugares donde habitan, y por tanto, desea mantener un trayecto donde no pueda ser alcanzado por tales cantos o hechizos. Además, sabe que siempre hay posibilidad de que se presenten tormentas en el camino y por tanto siempre quiere mantenerse a una distancia máxima D de posibles islas que puede usar como refugio. Dada una retícula que describe el mapa del área del planeta donde puede transitar Osideo, determina la cantidad mínima de celdas que debe recorrer Osideo para llegar de Yorta a Acati manteniéndose siempre fuera del alcance de las sirenas, de las brujas y a una distancia máxima D de una isla que pueda usar como refugio, considerando que si en el mapa hay una celda que represente un posible refugio, pero esta dentro del alcance de los cantos de las sirenas o hechizos de las brujas, entonces en realidad no puede ser considerada como refugio. Tanto Acati como Yorta son refugios válidos, es decir, son islas que cuentan como refugios y están fuera del alcance de sirenas y brujas. Por alguna extraña razón la embarcación de Osideo no puede moverse en diagonal en las celdas que describen el mapa, pero los cantos y hechizos si pueden moverse en diagonal.

Input

En el primer renglón hay dos números enteros N y M ($1 \leq N, M \leq 1000$) que representan el número de filas y columnas que tiene el mapa.

En el segundo renglón hay dos números enteros H ($1 \leq H \leq \max(N, M)$) y D ($1 \leq D \leq \left\lceil \frac{\max(N, M)}{2} \right\rceil$), que representan el alcance que tienen tanto el canto de las sirenas como los hechizos de las brujas, y la distancia máxima que puede viajar la embarcación a una isla donde se pueda refugiarse Osideo en caso de tormenta.

En los subsecuentes N renglones hay M caracteres que describen el mapa. Cada celda del mapa puede ser uno de los siguientes caracteres:

1. 'Y' - representa la ubicación de Yorta.
2. 'A' - representa la ubicación de Acati.
3. 'S' - representa una ubicación donde hay sirenas.
4. 'B' - representa una ubicación donde hay brujas.
5. 'R' - representa una isla donde pudiera refugiarse Osideo en caso de tormenta, siempre y cuando no este dentro del alcance de los cantos de sirenas o hechizos de brujas.
6. '.' - representa un espacio del mar por donde se puede mover la embarcación de Osideo.

Output

Un número entero que indica la cantidad mínima de celdas de mar que debe recorrer Osideo para llegar de Yorta a Acati cumpliendo con las restricciones que se indicaron previamente, o el texto **OSIDEO WILL DIE** en el caso en el que sea imposible llegar de Yorta a Acati.



Example

Input	Output
7 9 1 3 .Y..R....R.R.BB..R.S.....R. ...A....S	14
8 7 2 4A.B....RS Y..R...	OSIDEO WILL DIE

Use fast I/O methods



Problem E. Enarmonía

Source file name:	Enarmonia.c, Enarmonia.cpp, Enarmonia.java, Enarmonia.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/2/3 (C++/Java/Python) second(s) / 128 megabytes
Author(s):	Luis Daniel Carreño Pitalua - UFPS Cúcuta Colombia

Hoy, dos manuscritos reposan ante La Bruja del Drama y el Espectáculo, un ser que ha presenciado todas las historias jamás contadas. Sin embargo, observarlas desde afuera ya no le dice nada: lo que la fascina es intervenir en ellas, crear algo que ningún autor pudo proponer.

Ambos manuscritos narran una misma historia, pero desde distintas perspectivas, como si los dos autores hubieran vivido, o imaginado, los mismos eventos sin jamás saberlo. Cada manuscrito está dividido en actos, y cada acto guarda en silencio un **registro**: una propiedad emocional que el autor plasma de manera inconsciente al contar su historia. Para ella, los manuscritos no tienen fin, pues el último acto da paso al primero en un ciclo eterno.

Sin embargo, leer cada manuscrito por separado sería seguir el camino que otro trazó, sin espacio para lo inesperado. Por eso decide esta vez intervenir: tomando siempre el primer acto del primer manuscrito como punto de partida, construye una secuencia de exactamente T actos, eligiendo en cada momento de cuál de los dos tomar el siguiente. Puede continuar en el manuscrito actual o cambiar al otro, aunque el total de cambios no puede superar K . Cuando decide cambiar, el manuscrito que abandona queda pausado en el acto siguiente, listo para ser retomado exactamente donde lo dejó.

En esa lectura pueden surgir los **ecos**: momentos donde dos actos consecutivos comparten el mismo registro, sin importar de qué manuscrito provienen. Quizás la bruja los busca, quizás simplemente aparecen. Lo que sí es cierto es que su lectura siempre los encuentra. ¿Cuántos podría haber como máximo en esa lectura?

Input

La primera línea contiene dos enteros K y T ($0 \leq K < T$, $1 \leq T \leq 150$), el número máximo de cambios y la longitud de la lectura.

La segunda línea contiene un entero N_1 ($1 \leq N_1 \leq 150$), el número de actos del primer manuscrito, desde cuyo primer acto comienza siempre la lectura. Cada una de las siguientes N_1 líneas contiene un string binario de largo entre 1 y 7, representando el registro de cada acto.

La siguiente línea contiene un entero N_2 ($1 \leq N_2 \leq 150$), el número de actos del segundo manuscrito. Cada una de las siguientes N_2 líneas contiene un string binario de largo entre 1 y 7, representando el registro de cada acto.

Dado que los manuscritos son circulares, el acto siguiente al N_i -ésimo es nuevamente el primero. Se garantiza que todos los registros de ambos manuscritos tienen el mismo largo.

Output

Un único entero: la cantidad máxima de ecos en la lectura.



Example

Input	Output
0 6 4 0 1 1 1 5 0 1 0 0 0	2
5 6 4 01 00 00 01 4 00 11 00 10	3
0 5 1 11 5 01 01 01 01 01	4



Problem F. Flipando colores con la DIAN

Source file name:	Flipando.c, Flipando.cpp, Flipando.java, Flipando.py
Input:	Standard
Output:	Standard
Time / Memory limit:	4/6/6 (C++/Java/Python) second(s) / 256 megabytes
Author(s):	María Alexandra Velásquez Jaimes - UFPS Cúcuta Colombia

Hace unos años, Carlos perseguía problemas para resolverlos en competencias. Hoy surgió su problema más grande con la llegada de la vida adulta: pagarle a la DIAN.

A Carlos le llegó una notificación de la DIAN: tiene n trámites pendientes. La regla es simple pero tediosa, no se puede cerrar un trámite sin haber cerrado antes el trámite del que depende. Un trámite puede tener varios que dependen de él, pero cada uno depende siempre de uno solo.

Cerrar el trámite número i le toma p_i días entre filas y firmas. Y por supuesto, cada trámite acumula intereses de mora: por cada día que pase desde que Carlos empieza a hacer sus vueltas hasta que ese trámite queda cerrado, tiene que pagar w_i de interés. Es decir, si Carlos arranca el trámite en el día t y este le toma p_i días, paga $(t + p_i) \times w_i$ de interés. Y ojo: una vez que un trámite arranca, no se puede dejar a medias, hay que terminarlo antes de empezar el siguiente.

Carlos solo puede avanzar un trámite a la vez (todos están haciendo su declaración de renta y la DIAN no da abasto), y tiene que elegir el orden en que los cierra, respetando la burocracia antes descrita. Su objetivo es que el total de intereses pagados, entre los n trámites, sea lo más bajo posible.

A ti, como su contador de confianza, te toca calcular esa cifra: el costo mínimo posible que Carlos puede llegar a pagar, si elige el orden óptimo.

Input

La primera línea contiene un entero n ($1 \leq n \leq 10^6$), la cantidad de trámites.

La segunda línea contiene n enteros $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq 2000$), los días que toma cerrar cada trámite.

La tercera línea contiene n enteros $w_1, w_2, \dots, w_n$ ($1 \leq w_i \leq 2000$), el interés diario de mora de cada trámite.

La cuarta línea contiene un entero m ($0 \leq m \leq n - 1$), la cantidad de restricciones.

Cada una de las siguientes m líneas contiene dos enteros a_i, b_i ($1 \leq a_i, b_i \leq n$, $a_i \neq b_i$), indicando que el trámite a_i no puede cerrarse antes que el trámite b_i . Cada valor de a_i aparece a lo sumo una vez entre todas las restricciones, y las restricciones nunca forman un ciclo.

Output

Un único entero: el costo mínimo posible de la suma total de intereses de mora.



Example

Input	Output
6 2 1 3 1 4 1 1 5 1 2 1 3 4 2 1 3 2 5 4 6 5	72
8 8 5 9 3 10 6 10 7 4 7 3 2 1 4 1 2 7 2 1 3 1 4 2 5 2 6 3 7 3 8 7	549

Use fast I/O methods



Problem G. Guanex y el diámetro con actualizaciones

Source file name: Guanex.c, Guanex.cpp, Guanex.java, Guanex.py
Input: Standard
Output: Standard
Time / Memory limit: 1/2/2 (C++/Java/Python) second(s) / 128 megabytes
Author(s): Santiago Guaneme - IOI Colombia

Guanex en la últimas dos semanas ha estado preparando un buen reto de programación que hoy quiere compartir con todos en el marco del **UTP Open 2026**.

En el reto recibes como entrada un árbol de n nodos sobre el cual se tienen que realizar q modificaciones/-consultas. Las modificaciones/consultas son del tipo $x y$, que corresponde a crear un nuevo nodo con etiqueta x (donde se garantiza la etiqueta x no se ha utilizado previamente), siendo este el hijo del nodo y . Las modificaciones/consultas son persistentes, es decir, un nodo agregado permanece en el árbol y será tenido en cuenta en las siguientes modificaciones/consultas. El árbol original tiene un diámetro, el cual se puede ver afectado de consulta a consulta.

Input

La entrada contiene un único caso de prueba. La primera línea contiene un número entero n ($2 \leq n \leq 10^5$), el cual representa el número de nodos del árbol. Luego son presentadas $n - 1$ líneas, cada una de ellas con dos números enteros $u v$, que representan la arista que existe en el árbol entre los nodos u y v .

La siguiente línea contiene un número entero q ($1 \leq q \leq 10^5$), que representa el total de modificaciones/consultas que se realizarán sobre el árbol. Cada una de las siguientes q líneas contienen dos números enteros $x y$ que describen la modificación/consulta realizada sobre el árbol.

Nota 1: Se garantiza que las etiquetas de los nodos, a través de toda la ejecución son menores a $3 \cdot 10^5$, es decir, $0 \leq u, v, x, y < 3 \cdot 10^5$.

Nota 2: No se garantiza que el nodo con etiqueta 0 aparezca. Cada nodo y entregado ya existía al momento de hacer la query.

Nota 3: El diámetro corresponde a la mayor distancia entre cualquier par de nodos $u v$ en el árbol, donde $\text{dist}(u, v)$ corresponde a la cantidad de aristas en su camino único.

Output

La salida contiene $q + 1$ líneas, cada una de ellas conteniendo un número entero. La primera línea contiene el diámetro del árbol que se da como punto de partida. Las siguientes q líneas representan el diámetro del árbol después de cada una de la modificación/consulta que se realizan sobre el árbol.

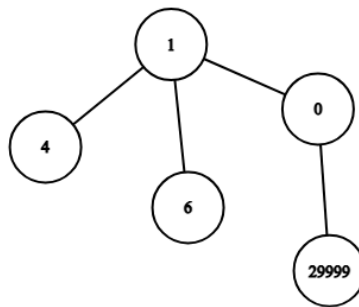
Example

Input	Output
5	3
0 1	4
1 4	5
1 6	5
299999 0	6
4	
5 4	
10 5	
9 6	
11 9	

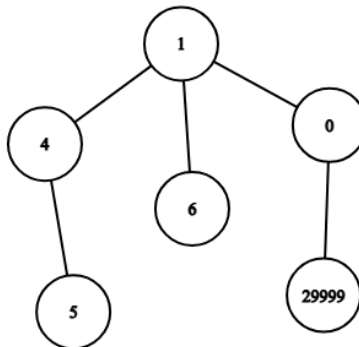
Use fast I/O methods

Explanation

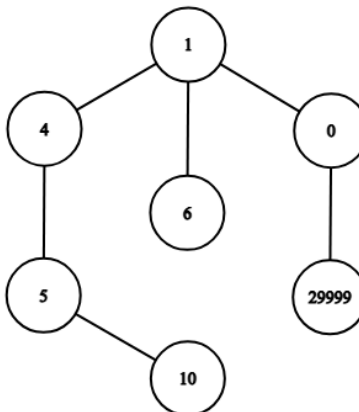
En el ejemplo del caso de prueba se tiene:



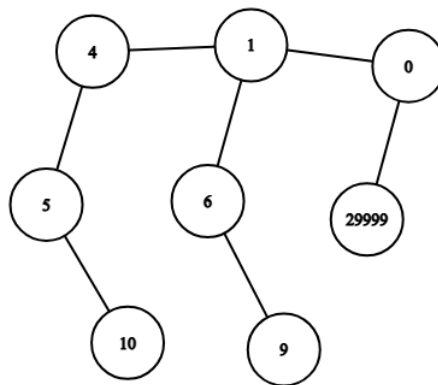
Árbol original con diámetro 3.



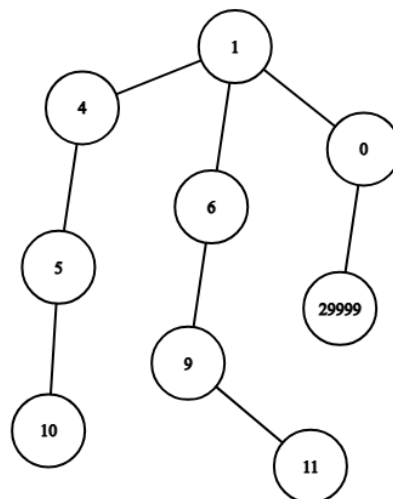
Árbol después de la primera modificación/consulta con diámetro 4.



Árbol después de la segunda modificación/consulta con diámetro 5.



Árbol después de la tercera modificación/consulta con diámetro 5.



Árbol después de la cuarta modificación/consulta con diámetro 6.

Problem H. Humbertov y su taza de café

Source file name:	Humbertov.c, Humbertov.cpp, Humbertov.java, Humbertov.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/5/2 (C++/Java/Python) second(s) / 64 megabytes
Author(s):	Hugo Humberto Morales Peña - UTP Colombia

El profesor Humbertov Moraloov tiene la fortuna de trabajar en una de las mejores Universidades del Eje Cafetero en Colombia. ¡Obviamente su bebida favorita para comenzar el día es una buena taza de café!, pero ... tiene una obsesión con respecto a mezclar el café con agua, donde debe quedar un 50% de café y un 50% de agua. Si no se respeta esto entonces el profesor considerará que el café esta muy aguado o muy espeso.

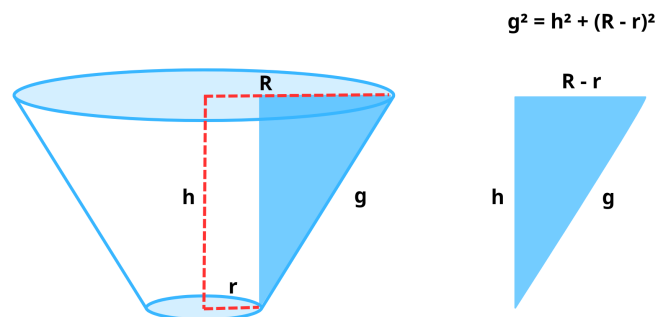
La taza de café del profesor Moraloov tiene la forma de un cono truncado volteado, con radio de la base r (o radio del fondo de la taza), radio de la parte superior R (o radio de la boca de la taza) y altura h .



Taza de café del profesor Humbertov Moraloov

El grupo de fanes del profesor Moraloov (los cuales se hacen llamar los **Moralovers**) se ha dado cuenta de la problemática que vive día a día su profesor con la taza de café y necesitan de tu ayuda para que por favor le indiques hasta donde debe de llenar de café su taza para que la combinación con el agua sea perfecta.

Los Moralovers te facilitan la siguiente información que podría llegar a ser de utilidad:



Cono Truncado

La fórmula para calcular el volumen del cono truncado es:

$$V = \frac{1}{3} \cdot \pi \cdot h \cdot (R^2 + r^2 + R \cdot r)$$

Input

La entrada comienza con un número entero t ($1 \leq t \leq 5 \cdot 10^4$) el cual representa el total de casos de prueba. Cada uno de los casos de prueba consta de tres números reales r R h ($1.5 \leq r \leq 4.0$, $2.5 \leq R \leq 6.0$, $7.0 \leq h \leq 10.0$, $r \leq R$), los cuales representan respectivamente el radio del fondo, el radio de la boca y la altura de la taza en centímetros.

Output

Por cada caso de prueba imprimir en una sola línea un número real que representa la altura hasta la cual el profesor Moraloov debe de llenar su taza de café para obtener la combinación perfecta. La altura (en



centímetros) será considerada correcta si el error absoluto o relativo no excede al 10^{-6} .

Example

Input	Output
4	5.271806416
2.5 3.8 8.8	6.509636245
2.0 4.0 10.0	5.561206129
1.5 2.5 9.0	4.400000000
3.0 3.0 8.8	

Use fast I/O methods

Explanation

Más que alguna explicación del reto son unas disculpas por utilizarlo hoy en el **UTP Open 2026**.

Como autor, este es uno de los retos de programación con el cual más me he divertido durante su construcción ... me reí mucho al escribir, acuñar y utilizar el termino de “los Moralovers” ... ¡en la vida uno debe disfrutar lo que hace!, pero ... no quiero que este reto se quede perdido al interior de las páginas de mi libro de *Estructuras de Datos y sus aplicaciones en Programación Competitiva - Tomo I*². Son ya 10 meses de la publicación del libro y el reto tiene cero envíos en la plataforma donde tengo subidos todos los retos que lo conforman. Por ese motivo, hoy tengo el gusto de compartir con todos ustedes el reto que se motivo por una historia “real” de mi día a día.

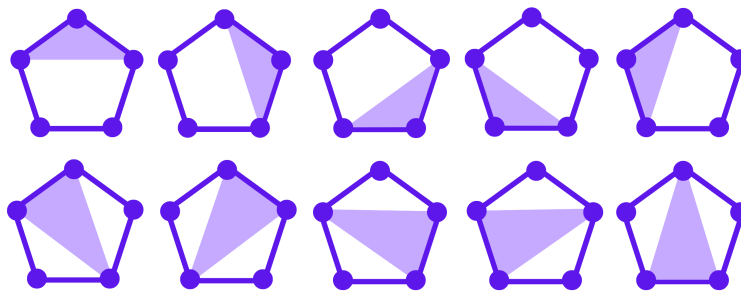
Hugo Humberto Morales Peña
Universidad Tecnológica de Pereira
Pereira, capital del Eje Cafetero en Colombia

²Mi libro está en **versión digital** con **acceso abierto** (para **descargar de forma gratuita**) desde la página de la **Editorial UTP** en el enlace: <https://hdl.handle.net/11059/16567>

Problem I. Internal triangles

Source file name: Internal.c, Internal.cpp, Internal.java, Internal.py
Input: Standard
Output: Standard
Time / Memory limit: 1/1/1 (C++/Java/Python) second(s) / 64 megabytes
Author(s): Gabriel Gutiérrez Tamayo - UTP Colombia

Dado el número de lados de un polígono regular convexo, determine cuántos triángulos distintos pueden formarse seleccionando tres de sus vértices (ver figura).



Todos los posibles triángulos del polígono convexo regular de 5 lados.

Un polígono es *convexo* si todos sus ángulos internos son menores de 180 grados.

Un polígono es *regular* si todos sus lados y ángulos internos son iguales.

Dos triángulos se consideran distintos si al menos uno de los vértices de un triángulo no pertenece al otro.

Input

La primera línea contiene un entero q ($1 \leq q \leq 10^5$), que indica el número de consultas. Cada una de las siguientes q líneas contiene un entero n ($3 \leq n \leq 10^{18}$).

Output

Para cada consulta, imprimir una línea conteniendo la respuesta correspondiente. Dado que la respuesta a cada consulta puede ser muy grande, imprimirla módulo $10^9 + 7$.

Example

Input	Output
4	1
3	4
4	10
5	35
7	

Use fast I/O methods



Problem J. Juan y sus ovejas

Source file name: Juan.c, Juan.cpp, Juan.java, Juan.py
Input: Standard
Output: Standard
Time / Memory limit: 2/3/4 (C++/Java/Python) second(s) / 128 megabytes
Author(s): Jorge Humberto Teran Pomier - UMSA Bolivia

Juan tiene N ovejas. Cada oveja tiene un arete con un código que la identifica, estos códigos comienzan en 1 y terminan en N . No hay códigos repetidos. Juan quiere determinar cuantas razas diferentes de ovejas tiene. Pero como no conoce mucho de razas, lo que decidió hacer es tomar pares de ovejas que el cree pueden ser de la misma raza y registrar sus códigos.

Por ejemplo, si Juan tiene 9 ovejas y registró los siguientes 6 pares de ovejas que tienen la misma raza:

(4, 7), (5, 3), (2, 5), (6, 8), (1, 5) y (4, 5)

Se tiene que los pares de ovejas (4, 7), (5, 3), (2, 5), (1, 5) y (4, 5) pertenecen a una misma raza. El par de ovejas (6, 8) es una raza diferente y la oveja 9 no esta en ningún par, por lo tanto, esa es la única oveja que pertenece a otra raza.

Se tienen 3 razas de ovejas diferentes y la raza que tiene más ovejas es 6.

Input

La entrada consiste de múltiples casos de prueba (como máximo 100). Cada caso de prueba consiste de varias líneas, la primera línea contiene dos números enteros no negativos N P ($1 \leq N \leq 10^5$, $0 \leq P \leq 2 \cdot 10^5$), que representan respectivamente el número de ovejas y la cantidad de pares de ovejas. Cada una de las siguientes P líneas en el caso de prueba contienen dos números enteros positivos u v ($1 \leq u, v \leq N$, $u \neq v$), que representan los identificados de las ovejas que tienen la misma raza.

La entrada finaliza con un caso de prueba que contiene 0 0, para la cual no se debe hacer nada, solamente finalizar el programa.

Output

Para cada caso de prueba se debe imprimir en una sola línea dos números enteros positivos R C , que representan respectivamente, el número de razas de las ovejas y la raza que más ovejas tiene.

Example

Input	Output
9 6 4 7 5 3 2 5 6 8 1 5 4 5 0 0	3 6

Use fast I/O methods

Problem K. K-th shortest path

Source file name:	Kthpath.c, Kthpath.cpp, Kthpath.java, Kthpath.py
Input:	Standard
Output:	Standard
Time / Memory limit:	2/2/2 (C++/Java/Python) second(s) / 128 megabytes
Author(s):	Hugo Humberto Morales & Gabriel Gutiérrez - UTP Colombia

El profesor Humberto Moralo está explorando nuevas fuentes de dinero diferentes a su labor docente. Por este motivo se le ha ocurrido una idea pensando en la seguridad de personajes de alto riesgo, como políticos, empresarios, multimillonarios, etc.

En Latino América, los secuestros o atentados están a la orden del día, por este motivo, no siempre se debe utilizar la misma ruta en los desplazamientos terrestres al interior de la ciudad. Por este motivo sugiere que se utilice el k -ésimo recorrido más corto en lugar de utilizar siempre el recorrido más corto, para de esta forma despistar a los enemigos.

El profesor Moralo tiene buenas ideas, pero no tiene tiempo para implementarlas, por ese motivo necesita de tu ayuda como estudiante del curso de Estructura de Datos para solucionar este problema.

El trabajo a realizar consiste en calcular el k -ésimo recorrido más corto que se pueden generar a partir de un grafo no dirigido con pesos en las aristas, donde el k -ésimo recorrido más corto **no puede incluir** alguna arista que pertenezca a alguno de los 1-ésimo, 2-ésimo, 3-ésimo, ..., $(k-1)$ -ésimo recorridos más cortos.

Input

La entrada contiene un único caso de prueba conformado de varias líneas. La primera línea del caso de prueba consta de cinco números enteros positivos $N M K S D$ ($7 \leq N \leq 10^4$, $0 \leq M \leq \min\left(\frac{N \cdot (N-1)}{2}, 10^5\right)$, $1 \leq K \leq 10$, $1 \leq S, D \leq N$, $S \neq D$), que representan el número de vértices, la cantidad de aristas, el número del recorrido óptimo a utilizar, vértice inicial (fuente) y el vértice final (destino), respectivamente. Cada una de las siguientes M líneas representan las aristas del grafo. Cada arista constará de tres números enteros positivos $V_1 V_2 P$ ($1 \leq V_1, V_2 \leq N$, $V_1 \neq V_2$, $1 \leq P \leq 10^8$), donde V_1 y V_2 son los números de los vértices y P es el peso de la arista.

Output

La primera línea en la salida debe contener un número entero no negativo R el cual representa la distancia mínima del k -ésimo recorrido óptimo desde el vértice inicial al vértice final. La segunda línea debe contener el recorrido a realizar desde el vértice inicial (S) hasta el vértice final (D). Para el formato de salida mirar el ejemplo.

Nota 1: Se garantiza que para los casos de prueba, el reto de programación tiene solución única.



Example

Input	Output
7 21 5 3 6	153
1 2 30	3 - 5 - 6
1 3 15	
1 4 10	
1 5 77	
1 6 63	
1 7 61	
2 3 83	
2 4 25	
2 5 60	
2 6 65	
2 7 40	
3 4 41	
3 5 71	
3 6 20	
3 7 86	
4 5 75	
4 6 45	
4 7 35	
5 6 82	
5 7 21	
6 7 31	

Use fast I/O methods

Problem L. Locate Toby's nest

Source file name:	Locate.c, Locate.cpp, Locate.java, Locate.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/1/2 (C++/Java/Python) second(s) / 128 megabytes
Author(s):	Hugo Humberto Morales Peña - UTP Colombia

Tobby es un Pinscher de cadanita, de muy mal carácter y con serios delirios de Rottweiler. **Pelusa** es una linda y hermosa gata de colores blanco y negro en su pelaje y de color verde en sus ojos ... Pelusa es una celebridad porque sale en todos los videos de las clases realizadas durante pandemia por el profesor Humbertov Moralo, ¡Pelusa no faltó a ninguna de las clases virtuales durante los cuatro semestres de la pandemia!, es más, para Pelusa la pandemia fue la oportunidad de tener en casa a su “papá humano” durante todo el día, todos los días ... realmente no todo durante la pandemia fue malo, y Pelusa da fe de ello.



Tobby y Pelusa “Moralov”. Imágenes generadas con ChatGPT en estilo “Studio Ghibli”

Por su belleza Pelusa tiene muchos pretendientes gatunos, pero ... Toby (su hermano perruno) no avala a ninguno de estos y no los deja ingresar a la casa, es más, si alguno logra ingresar lo corretea por la casa hasta lograr sacarlo.

Toby necesita ubicar su nido en un lugar estratégico, desde el cual pueda llegar a cualquiera de los sitios de la casa (es decir, a cualquier lugar al que se pueda llegar caminando) recorriendo la mejor distancia posible.

Por facilidad, usted puede considerar que la casa del profesor Moralo es una cuadrícula (una matriz) de celdas. Una celda puede ser una pared o puede ser un pasillo (es decir, una celda por la cual se puede caminar). En cada momento, cualquiera de los integrantes de la casa del profesor Moralo pueden moverse a una nueva celda desde su posición actual si ellas comparten un lado y las dos son celdas de pasillo.

Toby se ha dado cuenta que eres un estudiante del curso de programación y necesita de tu ayuda para poder resolver este problema, para lo cual debes escribir un programa para determinar la celda en la cual él debe poner su nido y desde el cual él vigilará que no ingresen los pretendientes gatunos de Pelusa.

Nota 1: Se garantiza que para cualquier par de celdas de pasillo al interior de la casa existe un único camino entre ellas.

Nota 2: Se garantiza que para cualquier celda de pasillo se puede llegar a cualquiera de las celdas de pasillo al interior de la casa del profesor Moralo.



Input

La entrada comienza con un entero positivo T ($1 \leq T \leq 10$), denotando el número de casos de prueba.

Cada caso comienza con una línea que contiene dos números enteros positivos H W (High (alto), Wide (ancho), $3 \leq H, W \leq 1000$). El caso de prueba continua con H líneas, cada una de las cuales contiene W caracteres. Cada carácter representa el estatus de una celda como sigue:

1. '.' - para indicar que es una celda de pasillo (celda por la cual se puede caminar).
2. '#' - para indicar que es una celda de pared.

Output

Para cada caso de prueba, imprima una línea con el siguiente formato **Case idCase: fila columna**, donde **idCase** se reemplaza por el número del caso de prueba en el cual se encuentra, **fila** se reemplaza por el número de la fila y **columna** se reemplaza por el número de la columna de la celda donde Toby debe ubicar el nido. Como se podrían llegar a tener varias celdas que cumplen, entonces escoger la que tenga el valor de la **columna** más pequeño, si aún se tienen varias celdas que cumplen, entonces entre estas escoger la que tenga el valor de la **fila** más pequeño. Para mayor claridad en el formato de entrada y salida mirar los ejemplos a continuación.

Example

Input	Output
3 10 8 ####.### #.....# #.#.#.## #.#.#..# #..###.. ##.#.##. ...#...# ##.#.#.# #...#.# #.####.# 6 5 #.### #.#.# #.#.# #.#.. ..##. 7 6 #..... ##.##. ...##. .##... .###.# .#.#.# ...#.#	Case 1: 5 2 Case 2: 2 3 Case 3: 1 3

Use fast I/O methods

Problem M. Marble tilt maze

Source file name:	Marble.c, Marble.cpp, Marble.java, Marble.py
Input:	Standard
Output:	Standard
Time / Memory limit:	1/1/1 (C++/Java/Python) second(s) / 128 megabytes
Author(s):	Samuel Leonardo Nacache Pérez - UCV Venezuela

Para calmar el alto nivel de estrés producido por el trabajo, Samuel incluyó en sus compras nerviosas de Amazonia un “laberinto de canicas”. Este divertido juego consiste de un tablero de madera de dimensiones $N \times M$ con paredes y agujeros en su superficie. El juego incluye exactamente dos canicas, y el objetivo es llevar ambas canicas a una región ganadora (marcadas con **G** en el tablero), evitando que las canicas entren por algún agujero o se salgan de los bordes del tablero.

Se puede asumir que cada una de las casillas del tablero mide 1 cm de alto y 1 cm de ancho, y las canicas poseen un diámetro de 1 cm.

Para mover las canicas, el tablero puede ser inclinado hacia adelante, atrás, izquierda o derecha. Al inclinar el tablero, ambas canicas se deslizarán al mismo tiempo en la dirección escogida. Antes de cambiar de dirección, el jugador debe estabilizar el tablero (es decir, sin inclinación alguna, perfectamente plano). Nótese que una o ambas canicas pueden permanecer en la misma posición si al inclinar el tablero éstas se chocan contra una pared o contra otra canica. Dos canicas **NO** pueden ocupar la misma posición en el tablero.

Para ganar el juego, ambas canicas deben estar en una posición del tablero marcada como ganadora. Samuel quiere ganar con la menor cantidad de movimientos posible, es decir que quiere inclinar el tablero la menor cantidad de veces.

Samuel se encuentra aun más estresado que antes de comprar el juego. ¿Puedes ayudarlo?

Input

La entrada inicia con dos números enteros N y M ($2 \leq N, M \leq 30$), las dimensiones del laberinto.

Las siguientes N líneas contendrán M caracteres, representando los elementos del laberinto. Cada posición (i, j) del tablero contendrá uno de los siguientes caracteres:

- ‘.’: Un espacio vacío.
- ‘#’: Una pared.
- ‘O’: Un agujero.
- ‘G’: Una posición ganadora.

La penúltima línea de la entrada tendrá dos enteros r_1, c_1 ($1 \leq r_1 \leq N, 1 \leq c_1 \leq M$), la posición de la primera canica en el tablero.

La última línea de la entrada tendrá dos enteros r_2, c_2 ($1 \leq r_2 \leq N, 1 \leq c_2 \leq M$), la posición de la segunda canica en el tablero.

Las canicas nunca estarán inicialmente en un agujero o en una pared, pero si pueden estar en una posición ganadora. Adicionalmente, puede que no haya ninguna posición ganadora en el tablero.

Output

La salida consiste de un único entero indicando la mínima cantidad de movimientos necesarios para ganar el juego. Si es imposible ganar, se debe mostrar -1.



Example

Input	Output
6 8 ##### #.....# #.#..#.# #.#..#.# #.#GG#.# ##### 5 2 5 7	4
2 4 ...G G... 1 1 2 4	-1
5 8 ##### #...GGG# #.0.#G.# ##G.#.## ##### 4 3 4 6	3